

Simulations of Trajectories in General Relativity

Alex Huff and Emily Troutman

May 15, 2026

1 Introduction

Our final project involves numerically calculating and plotting the trajectories of particles using the principals of General Relativity (GR). The core tenent of GR is that gravity is not a force, but rather the curvature of a 4-Dimensional spacetime. This key difference is what creates deviations in derived trajectories of particles between Newtonian gravity and GR; it also makes the computations significantly more intensive.

The fundamental equations governing General Relativity are the Einstein equations. In geometrized units where the speed of light (c) and Newton's gravitational constant (G) both equal 1, these equations are written as

$$R_{\mu\nu} - \frac{1}{2}Rg_{\mu\nu} + \Lambda g_{\mu\nu} = 8\pi T_{\mu\nu} \quad (1)$$

The left-hand side of this equation encodes the curvature of spacetime, whereas the right-hand side encodes the required mass-energy density for such a curvature. The fundamental variable (i.e. the variable one would solve for) of these equations is the metric tensor, $g_{\mu\nu}$. Although not explicit due to the tensor notations, the $R_{\mu\nu}$ and R terms both contain second partial derivatives of the metric. Solving the Einstein Field Equations (1) is extremely difficult to do because they are a set of 10 independent, highly nonlinear, hyperbolic, partial differential equations of which only a small number of analytical solutions exist. Luckily, our program assumes a metric is already known by the user so they can test its properties.

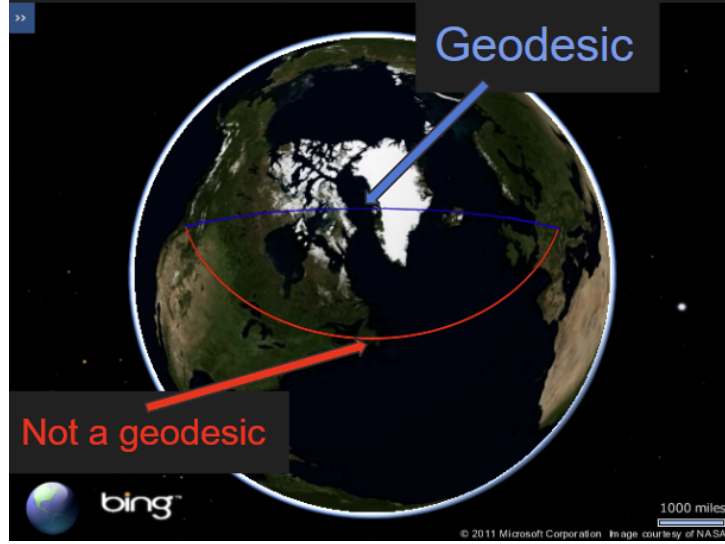


Figure 1: Example of a geodesic path and non-geodesic path on Earth.

The metric is important to know because it allows for the calculation of distances (and by extension angles) via defining an inner product for the curved space ¹:

$$\vec{u} \cdot \vec{v} = g_{\mu\nu} u^\mu v^\nu \quad (2)$$

Additionally, once a metric is known, one can use the principal of least action to derive the trajectories of particles within any spacetime. In GR, the proper time is defined as the "arc-length" of a path traced by a particle:

$$d\tau = -g_{\mu\nu} dx^\mu dx^\nu \quad (3)$$

Particles not under the influence of external forces will follow paths that extremize their proper time. If one does the proper calculations using variational calculus, they will obtain the geodesic equations:

$$\frac{d^2 x^\rho}{d\lambda^2} + \Gamma_{\mu\nu}^\rho \frac{dx^\mu}{d\lambda} \frac{dx^\nu}{d\lambda} = 0 \quad (4)$$

The solutions to these equations yield paths called geodesics.

A geodesic is essentially the "shortest" and "straightest" path between two points in a given space. The $\Gamma_{\mu\nu}^\rho$ in these equations are called the

¹In this paper, equations containing indices written as both a subscript and superscript in the same product represent summations over those indices.

Christoffel symbols, and while they have an infinite number of definitions, we chose to define them using the Levi-Civita definition:

$$\Gamma_{\mu\nu}^{\rho} = \frac{1}{2}g^{\rho\sigma}(\frac{\partial g_{\mu\sigma}}{\partial x_{\nu}} + \frac{\partial g_{\nu\sigma}}{\partial x_{\mu}} - \frac{\partial g_{\mu\nu}}{\partial x_{\sigma}}) \quad (5)$$

Any massive or massless particle will follow a geodesic path when no other forces act on them. Because gravity is not a force in GR, this means these particles still follow geodesic paths even when they are within a "gravitational field." Thus, in order for our program to calculate these general relativistic trajectories, it will need to build and numerically integrate the geodesic equations.

2 Methods

After a couple days of discussing potential project topics the team landed on a "build your own metric" idea. The goal of the script would be for a user to build their own spacetime metric and see the interesting point particle trajectories their metric produces. The most important component of this code would be that the math needs to work with user-inputted variables. In other words, the code could not assume things like specific coordinate systems, massive or massless particles, or the location of any singularities. Instead the mathematical variables within the Christoffel and geodesic equations need to be generated dynamically from the metric entered by the user.

Initially, on a white board we laid out the code as four main pieces:

1. Define the required user inputs for maximum user control.
2. Compute the Christoffel symbols from the metric.
3. Build and integrate the geodesic equations.
4. Plot the resulting trajectories with the user's coordinate system.

To get started on the actual coding portion we made a couple of python files and put them on a github titled CompPhysProject so both team members could work on the code on their own devices.

2.1 1. User Inputs

The first part of the code, right after the imports section, defines seven user-changeable inputs. These inputs tell the program what spacetime to use, what type of particle to simulate, where the particle starts, and how long the simulation should run.

The first input is the coordinate system. The user defines four symbolic coordinates, with the timelike coordinate listed first followed by three space-like coordinates. In the code, this section is written as

```
#INPUT: Please define the symbols of your coordinate system here,  
# make your timelike coordinate the FIRST symbol!  
coords = sp.symbols('x1 x2 x3 x4')  
  
for c in coords:  
    globals()[str(c)] = c
```

This loop above uses `sympy` to create symbolic variables for the user's coordinates. By turning the user-defined coordinates into symbolic variables, the mathematical functions that depend on those coordinates can be constructed, differentiated, and evaluated automatically.

The second input is whether the particle is massless or massive. Massive and massless particles follow different trajectories so it is important to distinguish between the two scenarios. In the code, this is written as

```
#INPUT: Please input if this particle is massless or not
massless_particle = True
```

If `massless_particle = True`, the code treats the particle as a photon. If `massless_particle = False`, the code treats the particle as massive.

The third input is the metric tensor $g_{\mu\nu}$. This is the main input of the build-your-own metric simulation. The user defines the metric as a 4×4 matrix, and the rows and columns must follow the same coordinate order as the user coordinate input:

$$x^\mu = (x_1, x_2, x_3, x_4).$$

In general, the metric has the form

$$g_{\mu\nu} = \begin{pmatrix} g_{11} & g_{12} & g_{13} & g_{14} \\ g_{21} & g_{22} & g_{23} & g_{24} \\ g_{31} & g_{32} & g_{33} & g_{34} \\ g_{41} & g_{42} & g_{43} & g_{44} \end{pmatrix}.$$

In the code, the user writes this as

```
#INPUT: Please define your metric here,
# make each column refer to above symbols in the same order!
```

```
g = sp.Matrix([
    [g11, g12, g13, g14],
    [g21, g22, g23, g24],
    [g31, g32, g33, g34],
    [g41, g42, g43, g44]
])
```

The metric input is flexible because the user is not limited to entering only simple numbers. The metric can include coordinate-dependent terms or functions of the coordinates. For example, the user could define a scale factor as a function of the timelike coordinate:

```
scale_factor = sp.exp(H*x1)
```

or define a coordinate-dependent expression:

```
Sigma = x2**2 + a**2*sp.cos(x3)**2
```

These quantities can then be used directly inside the metric matrix:

```
g = sp.Matrix([
    [-1, 0, 0, 0],
    [0, scale_factor**2, 0, 0],
    [0, 0, x2**2, 0],
    [0, 0, 0, x2**2*sp.sin(x3)**2]
])
```

This means the user can build a metric from symbolic coordinates and functions of those coordinates.

The fourth input tells the code whether to check if the metric is physically viable. In the code, this is written as

```
#INPUT: Please input if this metric should be physically viable.
physically_viable = True
```

If `physically_viable = True`, the code checks that the metric is symmetric, nonsingular, and has the correct Lorentzian signature. These checks help make sure the metric behaves like a valid spacetime metric before the simulation continues. The simulation works fine if `physically_viable = False` if someone wanted trajectories that are physically impossible.

The fifth input is the initial four-position of the particle. This is written in the same order as the coordinate system:

$$x^\mu(0) = (x^1(0), x^2(0), x^3(0), x^4(0)).$$

In the code, this is written as

```
#INPUT: Please input your initial 4-position,
# make each entry refer to coordinates in same order as above!
ri = [0, 0, 0, 0]
```

The exact physical meaning of these values depends on how the user defined the coordinate system.

The sixth input is the initial spatial velocity. The full four-velocity is

$$u^\mu = \frac{dx^\mu}{d\lambda} = (u^1, u^2, u^3, u^4),$$

where u^1 is the timelike velocity component and u^2 , u^3 , and u^4 are the spatial velocity components.

The user only inputs the spatial components:

$$(u^2, u^3, u^4).$$

In the code, this is written as

```
#INPUT: Please input your initial 3-velocity,
# follow same spatial order as above
vi = [0, 0, 0]
```

The code will later solve for the missing timelike component u^1 . This is necessary because the full four-velocity must satisfy the massive/massless normalization requirements.

$$g_{\mu\nu}u^\mu u^\nu = \begin{cases} -1, & \text{massive particle,} \\ 0, & \text{massless particle.} \end{cases}$$

By solving for u^1 , the code ensures that the initial velocity is consistent with both the metric and the type of particle being simulated.

The seventh input controls how long the simulation runs and how large each numerical step can be. In the code, this is written as

```
#INPUT: Please input the amount of "integration time"
# you would like this simulation to go by and the step amount
lamTot =
stepsize =
```

The variable `lamTot` sets the total range of affine parameter λ , the affine parameter is what the geodesic is integrated over. The variable `stepsize` sets the maximum step size used by the numerical solver (to be explained later in the methods).

2.2 2. Christoffel Symbol Calculator

Once the user inputted variables have been created, the next step the code makes is to compute the Christoffel symbols. The Christoffel symbols describe how the coordinates curve from different points around the spacetime. Christoffel symbols impact how the four velocity changes from point to point.

The code uses the Levi-Civita connection formula,

$$\Gamma_{\nu\rho}^{\mu} = \frac{1}{2}g^{\mu\sigma} \left(\frac{\partial g_{\sigma\nu}}{\partial x^{\rho}} + \frac{\partial g_{\sigma\rho}}{\partial x^{\nu}} - \frac{\partial g_{\nu\rho}}{\partial x^{\sigma}} \right).$$

This formula is implemented in the function `calcChristoffel`:

The function calculates the inverse metric,

$$g^{\mu\nu} = (g_{\mu\nu})^{-1},$$

using

```
ginv = g.inv()
```

This is necessary because the Levi-Civita connection formula uses $g^{\mu\sigma}$, the inverse metric.

Next, the code creates an empty $4 \times 4 \times 4$ array to store all Christoffel symbols:

```
christoffel = sp.MutableDenseNDimArray.zeros(4, 4, 4)
```

This array is three-dimensional because each Christoffel symbol has three indices:

$$\Gamma_{\nu\rho}^{\mu}.$$

Since each index can take four possible values, the code must calculate

$$4 \times 4 \times 4 = 64$$

Christoffel symbol components.

The nested loops go through every possible combination of μ , ν , and ρ :

$$\mu = 0, 1, 2, 3, \quad \nu = 0, 1, 2, 3, \quad \rho = 0, 1, 2, 3.$$

Here are the loops represented in the code:

```
for mu in range(0 , 4):
    for nu in range(0 , 4):
        for rho in range(0 , 4):
```

The outer three loops choose which component $\Gamma_{\nu\rho}^{\mu}$ is being calculated. The inner σ loop performs the required summation over σ . The three `sp.diff` terms calculate

$$\frac{\partial g_{\sigma\nu}}{\partial x^{\rho}} + \frac{\partial g_{\sigma\rho}}{\partial x^{\nu}} - \frac{\partial g_{\nu\rho}}{\partial x^{\sigma}}.$$

The code then multiplies this quantity by the inverse metric component $g^{\mu\sigma}$ and by the factor 1/2.

```
    for sigma in range(0 , 4):
        expr += .5 * ginv[mu , sigma] * (
            sp.diff(g[sigma , nu] , indices[rho]) +
            sp.diff(g[sigma , rho] , indices[nu]) -
            sp.diff(g[nu , rho] , indices[sigma])
        )
```

The christofel value is then saved in a list.

```
christoffel[mu , nu , rho] = expr
```

After the symbolic Christoffel symbols are calculated, the code converts them into numerical functions using

```
sp.lambdify(coords, christoffel[mu, nu, rho], "numpy")
```

We need to convert the Christoffel symbols into numerical objects so that the integrator can transform these functions into a particle trajectory. The symbolic expressions are useful for taking derivatives, but numerical functions are faster and easier to evaluate during integration.

2.3 3. Building and Integrating the Geodesic Equation

After calculating the Christoffel symbols, the code uses them to build the geodesic equation,

$$\frac{d^2x^\mu}{d\lambda^2} + \Gamma_{\nu\rho}^\mu \frac{dx^\nu}{d\lambda} \frac{dx^\rho}{d\lambda} = 0.$$

Using the four-velocity,

$$u^\mu = \frac{dx^\mu}{d\lambda},$$

this second-order equation can be rewritten as two first-order equations:

$$\frac{dx^\mu}{d\lambda} = u^\mu,$$

and

$$\frac{du^\mu}{d\lambda} = -\Gamma_{\nu\rho}^\mu u^\nu u^\rho.$$

This form is good for our purposes because numerical solvers like `solve_ivp` solve systems of first-order differential equations.

Before integrating, the code first solves for the missing timelike component of the four-velocity, satisfying the following conditions:

$$g_{\mu\nu} u^\mu u^\nu = \begin{cases} -1, & \text{massive particle,} \\ 0, & \text{massless particle.} \end{cases}$$

The code sets this condition using

```
if is_null:
    condition = 0
else:
    condition = -1
```

Then it defines the unknown timelike velocity component and builds the four-velocity:

```
ut = sp.symbols("ut", positive = True)
u = [ut, v[0], v[1], v[2]]
```

The code mathematically solves for the timeline input of the four-velocity by calculating $g_{\mu\nu}u^\mu u^\nu$ and solves for `ut`:

```
expr = 0
for m in range(0 , 4):
    for n in range(0 , 4):
        expr += u[m] * u[n] * numeric_g[m , n]

inner_product_constraint = sp.Eq(expr , condition)
time_part = sp.solve(inner_product_constraint , ut)
```

This gives an initial four-velocity that is consistent with the metric and with the particle type.

Next, the code creates the state vector that will be passed into the integrator:

$$\mathbf{y} = (x^0, x^1, x^2, x^3, u^0, u^1, u^2, u^3).$$

In the code, this is

```
initial_state = [cur_pos[0], cur_pos[1], cur_pos[2], cur_pos[3],
                 cur_vel[0], cur_vel[1], cur_vel[2], cur_vel[3]]
```

The first four entries are the particle's position, and the last four entries are the particle's velocity.

Inside the integration function, the code evaluates the Christoffel symbols by calling the Christoffel symbol function at the particle's current position. Then it uses the geodesic equation to calculate the acceleration terms:

$$\frac{du^k}{d\lambda} = - \sum_{\mu=0}^3 \sum_{\nu=0}^3 \Gamma_{\mu\nu}^k u^\mu u^\nu.$$

This is the key part of the code:

```
accel_list = []
for k in range(0 , 4):
    accel = 0
    for mu in range(0 , 4):
        for nu in range(0 , 4):
            accel += -1 * numeric_christos[k , mu , nu] * u[mu] * u[nu]
    accel_list.append(accel)
```

Finally, the function returns the derivative of the state vector:

```
return [u[0], u[1], u[2], u[3],  
        accel_list[0], accel_list[1], accel_list[2], accel_list[3]]
```

This return statement tells the solver how the state changes at each step: the coordinates change according to the four-velocity, and the four-velocity changes according to the Christoffel symbols.

The system is integrated using

```
sol = solve_ivp(  
    fun = change_of_state,  
    t_span = (0 , runtime),  
    y0 = initial_state,  
    method = 'Radau',  
    max_step = stepsize  
)
```

We chose the Radau method because geodesic equations can become numerically stiff for some metrics and initial conditions. A stiff differential equation is one where some parts of the solution change very quickly while others change slowly, making the solver take very small steps to stay stable. This can happen near strong-field regions, near coordinate singularities, or when the Christoffel symbols change rapidly (all of which occur for space-times around black holes).

Earlier versions of the code used more explicit methods, including hard-coded RK4 and then RK45. These methods worked for simpler test cases, but they were less reliable when the metric produced stiff behavior aka containing a black hole. Radau is an implicit solver designed for stiff systems, so it was a better choice for making the simulation more stable across a wider range of user-inputted metrics.

The numerical solver returns arrays containing the values of all coordinates and position components throughout the simulation. These are saved into an .npz file. The filename is created using the current date and time:

```
now = datetime.now()  
file_timestamp = now.strftime("%Y%m%d_%H%M%S")  
FILENAME = os.path.join(f"geodesicOutput_{file_timestamp}.npz")
```

The date and time stamp package is useful because it prevents simulations overwriting each other.

The code saves the trajectory information inside the npz file via this loop:

```
np.savez(  
    FILENAME,  
    t = sol.t,  
    y = sol.y,  
    coords = [str(c) for c in coords]  
)
```

Here, `sol.t` stores the affine parameter values, `sol.y` stores the position and velocity values from the integration, and `coords` stores the coordinate names as strings.

This separation is useful because the simulation and plotting do not need to be run together every time. Once a trajectory is calculated and saved, the same `.npz` file can be loaded later to make many different plots without rerunning the full geodesic calculation and users can use the npz information to make their own plots.

2.4 4. Plotting the Trajectories

The second part of the project is the plotting script. The plotting script loads the saved `.npz` trajectory file from the integration code. Before running the plotting section of the script, which can be ran independently from the trajectory script, the user first provides two plotting inputs:

```
is_angular = [0 , 1 , 1]
```

The list `is_angular` tells the code which spatial coordinates are angular. A value of 1 means the coordinate is angular, while a value of 0 means the coordinate is length-like.

The variable `data_path` tells the plotting script which saved trajectory file to open.

```
data_path = "geodesicOutput_20260429_233923.npz"
```

The plotting function begins by loading the saved data:

```
data = np.load(data_path , allow_pickle = True)
```

```
t = data['t']
y = data['y']
coords = data['coords']
```

Here, `t` stores the affine parameter values, `y` stores the state vector values, and `coords` stores the coordinate names. In other words, the plotting script extracts

$$\lambda, \quad x^\mu(\lambda).$$

Because some coordinates are angular, the code wraps angular coordinates so that they stay within one full period:

```
for i in range(3):
    if is_angular[i] == 1:
        nparray = np.array(y[i + 1], dtype=float)
        y[i + 1] = nparray % (2 * np.pi)
```

This applies the transformation

$$x_{\text{angular}} \rightarrow x_{\text{angular}} \bmod 2\pi.$$

This is useful because angular coordinates are periodic, so values separated by 2π represent the same angular location.

The code first creates coordinate plots showing how each coordinate changes with respect to the affine parameter:

$$x_0(\lambda), \quad x_1(\lambda), \quad x_2(\lambda), \quad x_3(\lambda).$$

It also plots the spatial coordinates against the timelike coordinate:

$$x_2(x_1), \quad x_3(x_1), \quad x_2(x_3).$$

This is set up using a grid of subplots:

```
fig1 , ax1 = plt.subplots(nrows = 4 , ncols = 2 , figsize = (14 , 14))
fig1.suptitle("Coordinates vs. Affine Parameter")
```

Each row corresponds to one coordinate. The left column plots the coordinate against the affine parameter λ , while the right column plots the coordinate against the timelike coordinate x_1 . For example, one spatial coordinate is plotted with

```
ax1[1,0].plot(t , y[1])
ax1[1,0].set_xlabel("lambda")
ax1[1,0].set_ylabel(f"{coords[1]}")

ax1[1,1].plot(y[0] , y[1])
ax1[1,1].set_xlabel(f"{coords[0]}")
ax1[1,1].set_ylabel(f"{coords[1]}")
```

This corresponds to plotting

$$x_1(\lambda) \quad \text{and} \quad x_1(x_0).$$

The plotting script then creates phase plots of the spatial coordinates. These plots compare spatial coordinates directly against each other:

$$x_1 \text{ vs. } x_2, \quad x_2 \text{ vs. } x_3, \quad x_3 \text{ vs. } x_1.$$

The code loops through each pair of spatial coordinates:

```
for i in range(0 , 3):
    for j in range(i + 1 , 3):
```

For each pair, the code checks whether one coordinate is angular and the other is not:

```
ang_i = is_angular[i]
ang_j = is_angular[j]

use_polar = (ang_i + ang_j == 1)
```

If exactly one coordinate is angular, the code makes a polar plot:

```
if use_polar:
    ax = fig2.add_subplot(3 , 3 , plot_num , projection = "polar")
```

The angular coordinate is treated as the angle, and the length-like coordinate is treated as the radius:

```
if ang_i == 1:
    theta_vals = y[i + 1]
    r_vals = y[j + 1]
else:
    theta_vals = y[j + 1]
    r_vals = y[i + 1]

ax.scatter(theta_vals , r_vals , c = t , cmap = "plasma" , s = 6)
```

If the coordinate pair does not require a polar plot, the code makes a regular scatter plot:

```
else:
    ax = fig2.add_subplot(3 , 3 , plot_num)
    ax.scatter(y[i + 1] , y[j + 1] , c = t , cmap = "plasma" , s = 6)
```

The points are colored by the affine parameter, so the color gradient shows how the particle moves along the path as the affine parameter increases. A colorbar is added using

```
sm = plt.cm.ScalarMappable(cmap = "plasma")
sm.set_array(t)
fig2.colorbar(sm , ax = fig2.axes , label = "Affine Parameter")
```

2.5 Overall Code Design

The code was built by first identifying the physics problem and then translating that problem into computational steps. The physics problem is to solve

$$\frac{d^2 x^\mu}{d\lambda^2} + \Gamma_{\nu\rho}^\mu \frac{dx^\nu}{d\lambda} \frac{dx^\rho}{d\lambda} = 0$$

for a user-defined metric. The computational problem is to turn this second-order equation into a first-order system that can be integrated numerically.

The final structure of the code is as follows:

1. The user defines the spacetime metric $g_{\mu\nu}$ along with several initial conditions.
2. The code computes $g^{\mu\nu}$ and $\Gamma_{\nu\rho}^{\mu}$.
3. The code constructs the geodesic equations and outlines the geodesic update equations.
4. The code solves and updates the geodesic equations numerically.
5. The code saves and plots the resulting trajectory.

This approach makes the code modular. Each function has a specific purpose: one function checks if the metric is physically viable, one function calculates Christoffel symbols, one function finds the initial time component of the four-velocity, one function integrates the geodesic equations, and a separate script handles plotting.

3 Results

In this section, we will highlight some key results we obtained when using our simulation. Note that the initial conditions and results are all given in geometrized units where $c = G = 1$.

3.1 Minkowski Metric - Flat Spacetime

The simplest possible metric with which to test our trajectory simulations was with flat spacetime. This metric is called the Minkowski Metric, and in cartesian coordinates $\{t, x, y, z\}$ it is written as

$$g_{\mu\nu} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Figure 2 shows a massive particle moving through flat spacetime with an initial position of $[t = 0, x = 0, y = 0, z = 0]$ and initial spatial velocity of $[v_x = -.2, v_y = .5, v_z = 0]$. Figure 3 shows a massless particle moving through flat spacetime with an initial position of $[t = 0, x = 0, y = 0, z = 0]$ and initial spatial velocity of $[v_x = 1, v_y = 0, v_z = 0]$. The most significant detail to notice in these simple simulations is the plots of spatial coordinates vs. t . For the massless particle, the slope of the x vs. t plot has a magnitude of one, which indicates that the particle is traveling at the speed of light. The paths that particles traveling at the speed of light follow are called Null Lines. In contrast, the x vs. t and y vs. t plots for the massive particle both have slopes with magnitudes less than one. Massive particles can never travel faster than light and must always travel slower. The paths that massive particles are said to be timelike because the starts and ends of these lines remain in causal contact.

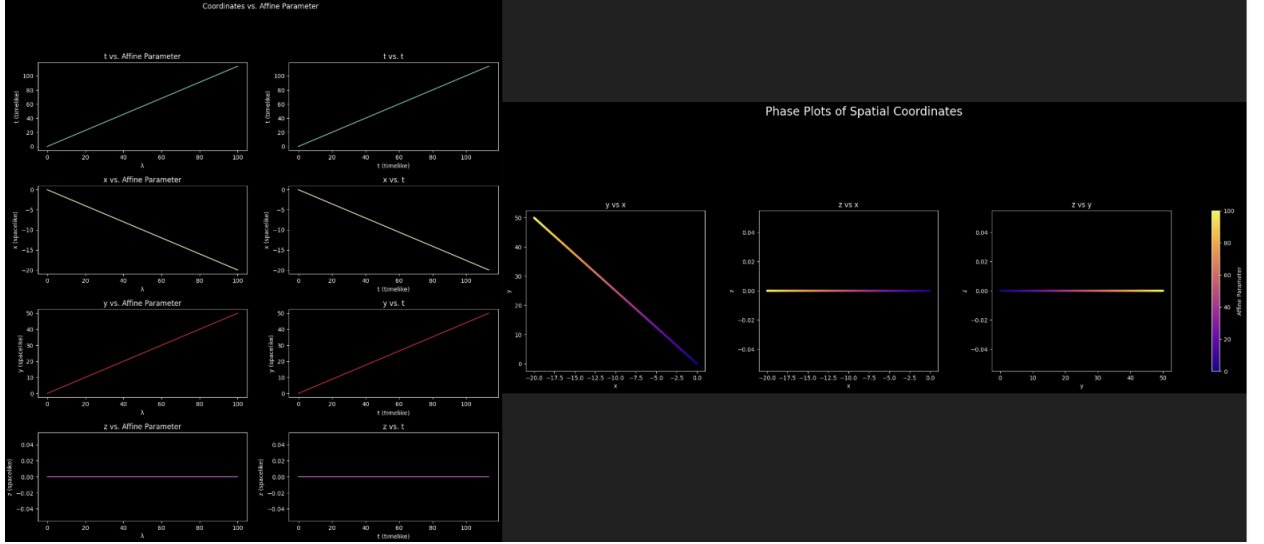


Figure 2: A massive particle moving in the Minkowski Metric.

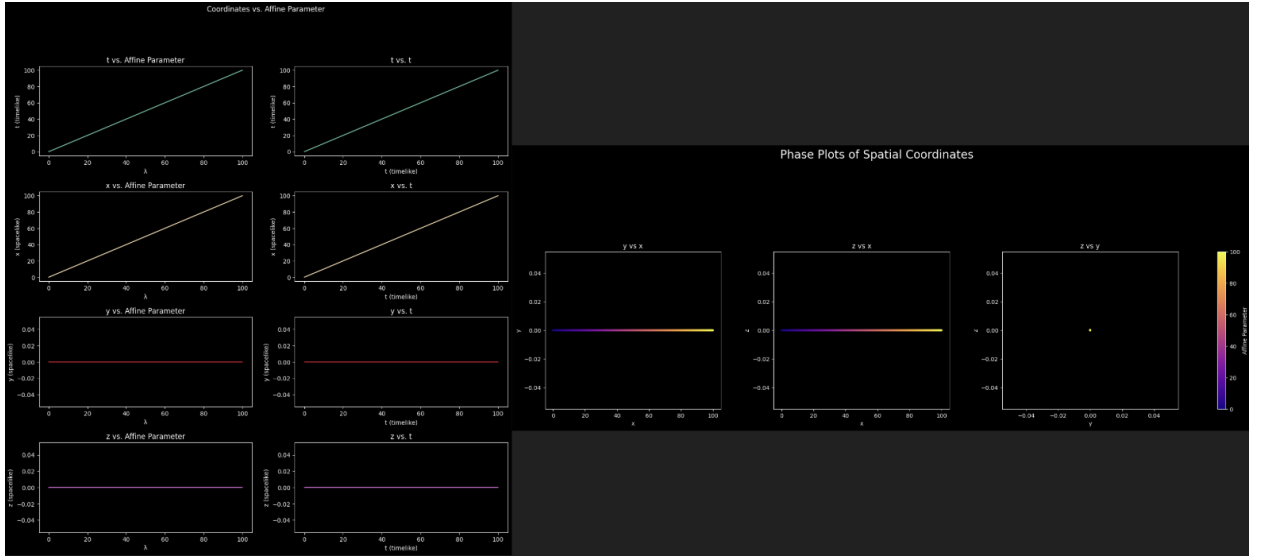


Figure 3: A massless particle moving in the Minkowski Metric.

3.2 Schwarzschild Metric - Nonrotating, Electrically Neutral Black Hole Spacetime

The Schwarzschild metric describes the spacetime outside of a nonrotating, electrically neutral black hole. It is a spherically symmetric and in areal coordinates $\{t, r, \theta, \phi\}$ is written as

$$g_{\mu\nu} = \begin{bmatrix} -(1 - \frac{2M}{r}) & 0 & 0 & 0 \\ 0 & \frac{1}{1 - \frac{2M}{r}} & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin^2(\theta) \end{bmatrix} \quad (7)$$

Using our program, we were able to investigate several key trajectories for both massive and massless particles. For all simulations, we used a Schwartzchild metric with a mass M of 5.

3.2.1 Massive Particle Trajectories

When investigating the trajectories of massive particles, one can use symmetries of the Schwartzchild metric to define conserved quantities. With these conserved quantities, it is possible to make an effective potential of the "gravitational field":

$$\mathcal{U}_{eff} = -\frac{M}{r} + \frac{\ell^2}{2r^2} - \frac{M\ell^2}{r^3} \quad (8)$$

In this equation, M is the mass of the black hole (which we said previously will be 5 for these Schwarzschild simulations) and ℓ is the angular momentum of the particle. Note that the first two terms of this equation are exactly what one would find if they used Newtonian gravity (the first is the gravitational potential proportional to r^{-1} and the second is the centrifugal potential caused by the frame of reference being noninertial). GR only adds an extra correcting term proportional to r^{-3} .

The first trajectory we can simulate is that of a scattering orbit using an initial position of $[t = 0, r = 100, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[v_r = -.07, v_\theta = 0, v_\phi = .0029]$. This orbit is shown in figure 4.

A more interesting trajectory we can simulate is an actual bound orbit. To create this orbit, we used an initial position of $[t = 0, r = 60, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[v_r = 0, v_\theta = 0, v_\phi = .005]$; the result of this

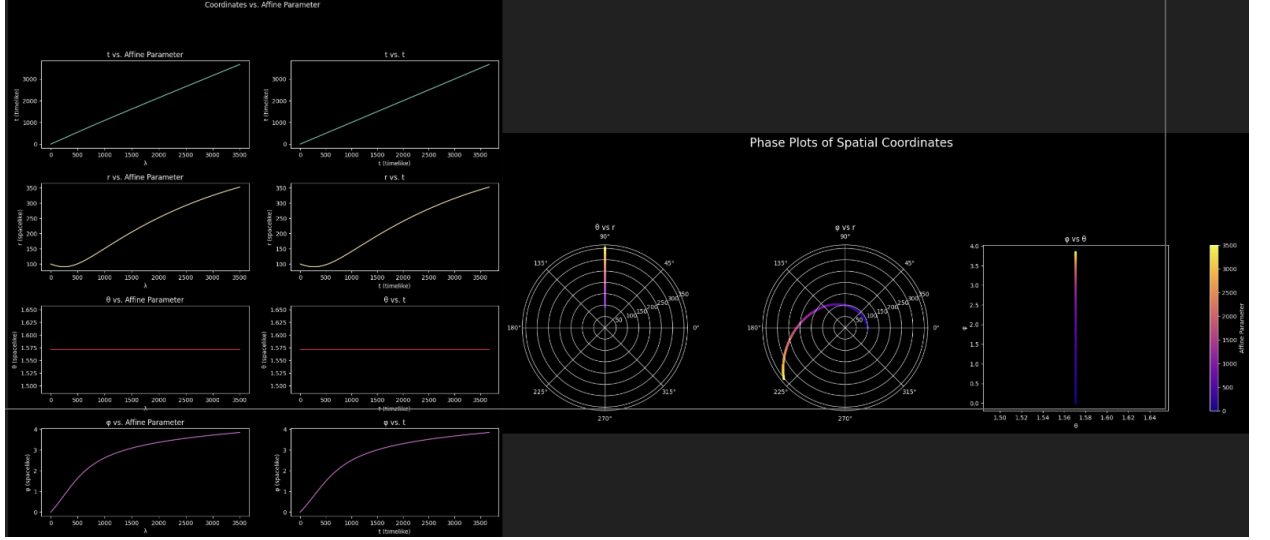


Figure 4: A massive particle undergoing a scattering trajectory in the Schwarzschild Metric.

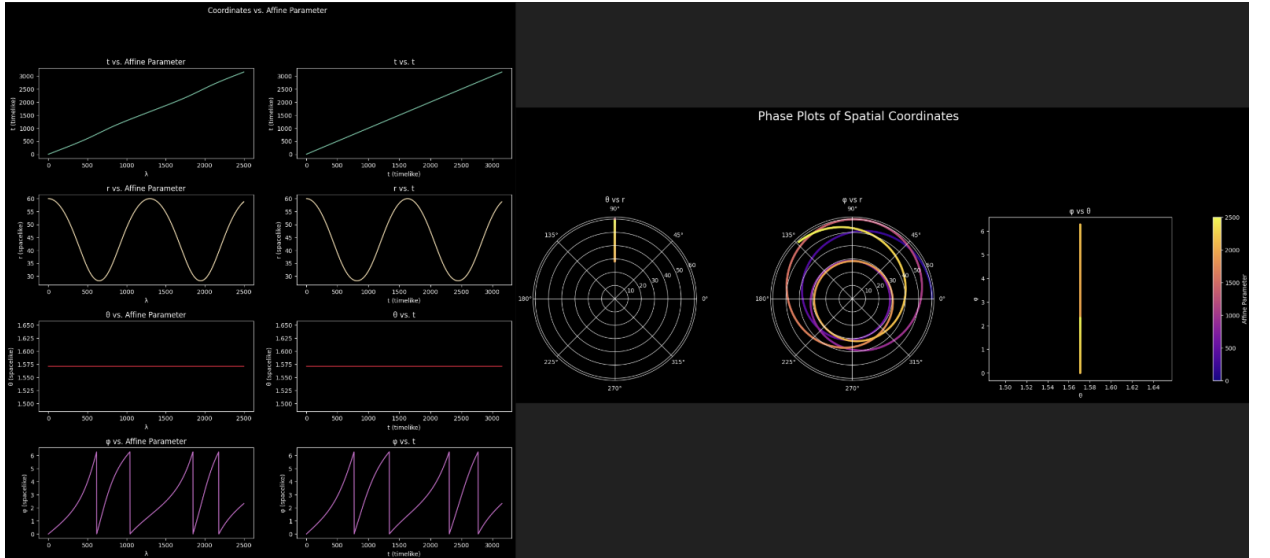


Figure 5: A massive particle within a bound orbit in the Schwarzschild Metric. Note that this orbit is precessing.

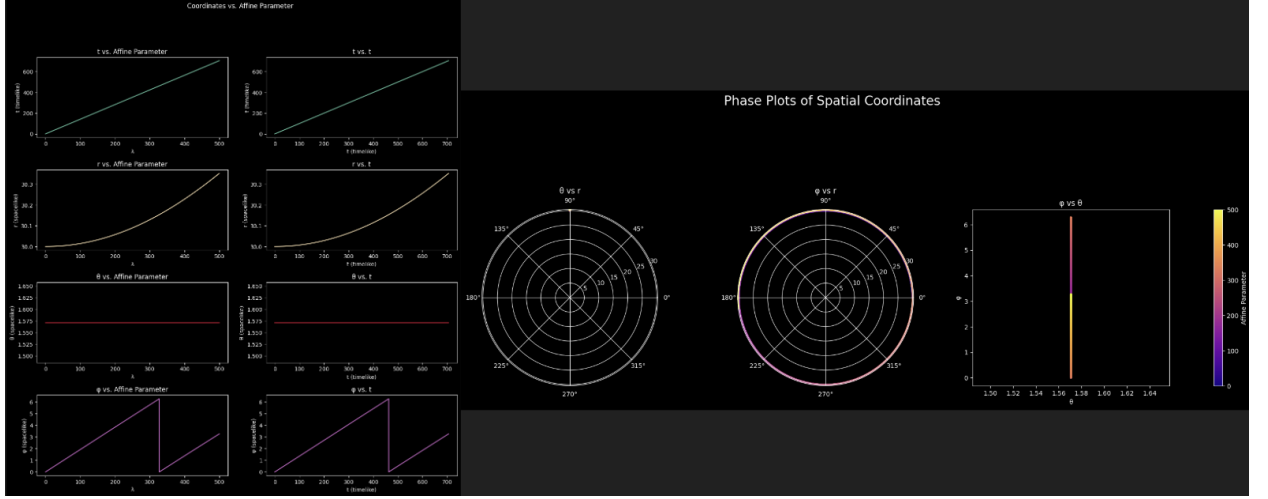


Figure 6: A massive particle within a circular orbit at the ISCO in the Schwarzschild Metric.

simulation is shown in figure 5. An interesting component of this orbit is that it precesses. In Newtonian gravity, the gravitational potential is proportional to r^{-1} . This type of potential is one of only two that conserves the Runge-Lenz vector (the other potential being that of the simple harmonic oscillator). When the Runge-Lenz vector is conserved, the orbit of the particle remains closed. The addition of the $-\frac{M\ell^2}{r^3}$ term due to GR means that the Runge-Lenz vector is no longer conserved, and the orbit precesses, the dynamics of which our program was able to capture.

In Newtonian gravity, a particle can obtain a stable circular orbit at any radius. The additional term in the effective potential due to GR actually causes there to be an inner most stable circular orbit (ISCO). Because our black hole in this simulation has a mass of 5, then the radius will be found at $6M = 6(5) = 30$. We were able to obtain a circular orbit in the ISCO using an initial position of $[t = 0, r = 30, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[v_r = 0, v_\theta = 0, v_\phi = .01925]$; this ISCO simulation is shown in figure 6.

There is an extra possible orbit for massive particles that exists in GR but not Newtonian gravity. In Newtonian gravity, the centrifugal potential $\frac{\ell^2}{2r^2}$ prevents particles with any amount of angular momentum from being captured (assuming you have two point particles). The extra term in the effective potential added by GR creates a minimum angular momentum below

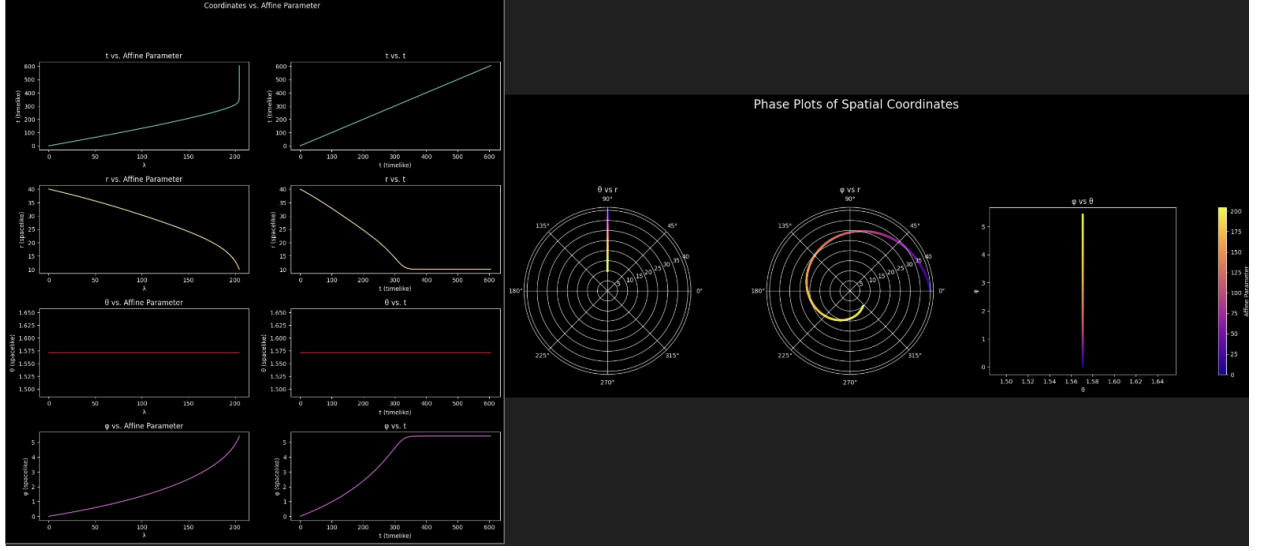


Figure 7: A massive particle undergoing a capture trajectory in the Schwarzschild Metric.

which the particle is actually captured by the "gravitational field." To create a capture orbit, we used an initial position of $[t = 0, r = 40, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[v_r = -.08, v_\theta = 0, v_\phi = .0105]$; this capture simulation is shown in figure 7. An interesting detail of this trajectory that our program was able to model was the effects of time dilation as the particle fell into the black hole. The plot of r vs. the affine parameter λ essentially shows what the particle itself would report as its position over time using co-moving clocks. On the other hand, the plot of r vs. t shows what an outside observer far from the curved spacetime would see as the particle falls into the black hole. At $r = 2M$, the g_{rr} term in the Schwarzschild metric becomes undefined. This discontinuity represents the event horizon of the black hole: any particle that passes this point can never get into causal contact with anything outside $r = 2M$ ever again. Thus, the outside observer can never actually see the particle cross $2M$ and instead sees the particle approach the black hole slower and slower, asymptotically approaching $2M$.

3.2.2 Massless Particle Trajectories

Because gravity in GR is just the curvature of spacetime, it is possible for particles with no mass (like photons) to also be effected by a gravitational field.

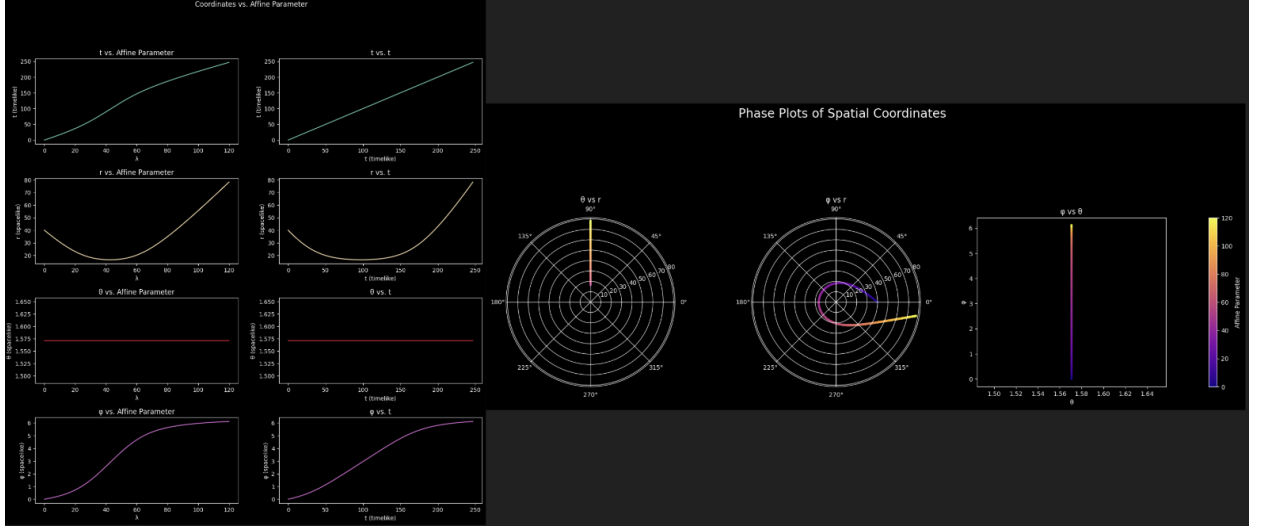


Figure 8: A massless particle undergoing a scatter trajectory in the Schwarzschild Metric.

It is actually also possible to construct an effective "gravitational potential" for photons using the quantities that are conserved for massless particles:

$$\mathcal{W}_{eff} = \frac{1}{r^2} - \frac{2M}{r^3} \quad (9)$$

The first photon trajectory we simulated was a scattering orbit. This effect is sometimes called "gravitational lensing." To make such an orbit, we used an initial position of $[t = 0, r = 40, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[-1, 0, .02]$; this scatter simulation is shown in figure 8.

If one finds the extrema of $\mathcal{W}_{eff}$ with respect to r , they will find one local maximum, located at $3M$. This maximum represents the only possible orbit for a photon in this metric, which is sometimes called the "photon sphere." To simulate a photon in this orbit, we used an initial position of $[t = 0, r = 15, \theta = \pi/2, \phi = 0]$ and initial spatial velocity of $[v_r = 0, v_\theta = 0, v_\phi = .0344]$ and is shown in figure 9.

Because the local extrema of $\mathcal{W}_{eff}$ is a maximum, it means the photon sphere orbit is unstable. A slight modification to the initial conditions will cause the photon to either spiral into or out of the black hole. To showcase this instability, we ran a simulation with the same initial spatial velocity as figure 9 but with an initial r value of 14.999 instead of 15. This new

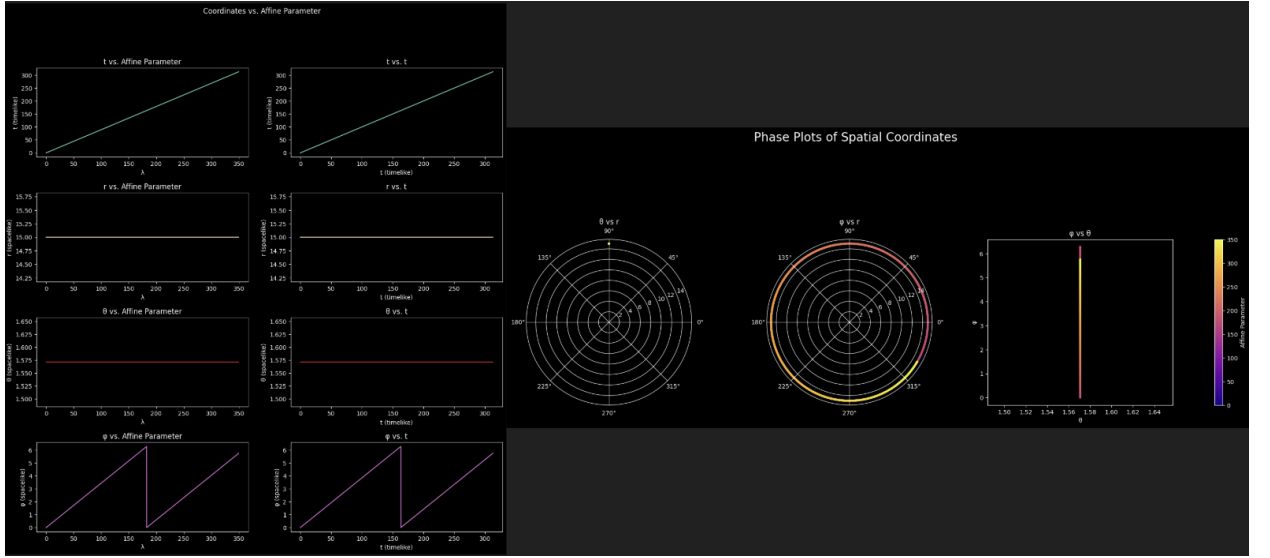


Figure 9: A massless particle within a circular orbit at the photon sphere in the Schwarzschild Metric.

simulation is shown in figure 10 Even though the initial r values between these two simulations is only .001, the dynamics completely change from a bound orbit to a capture trajectory.

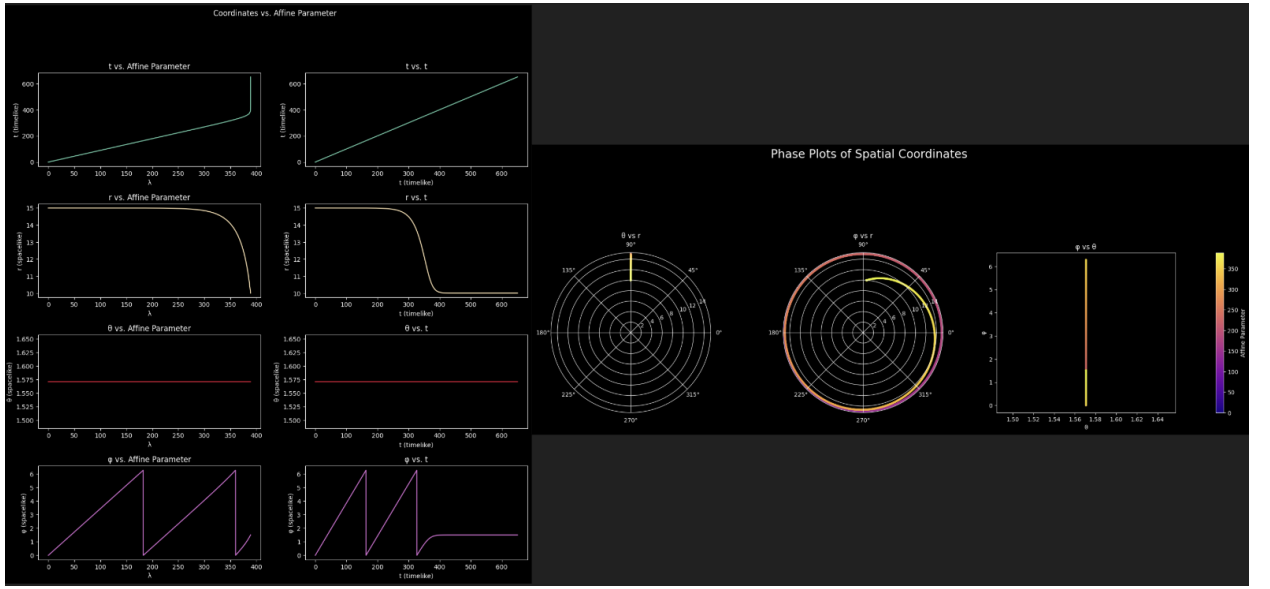


Figure 10: A massless particle undergoing a capture trajectory in the Schwarzschild Metric. Notice how just a slight modification to the photon sphere initial conditions caused the photon to dive into the black hole rather than orbit it.

3.3 Kerr Metric - Rotating, Electrically Neutral Black Hole Spacetime

The Kerr metric describes the spacetime outside of a rotating, electrically neutral black hole. Unlike the Schwarzschild metric, the Kerr metric is not spherically symmetric because the black hole has angular momentum. Instead, it is axially symmetric, meaning the spacetime has a preferred axis of rotation. In Boyer-Lindquist-like coordinates $\{x_1, x_2, x_3, x_4\}$, where x_1 is the timelike coordinate and x_4 is the azimuthal angular coordinate, the Kerr metric can be written as

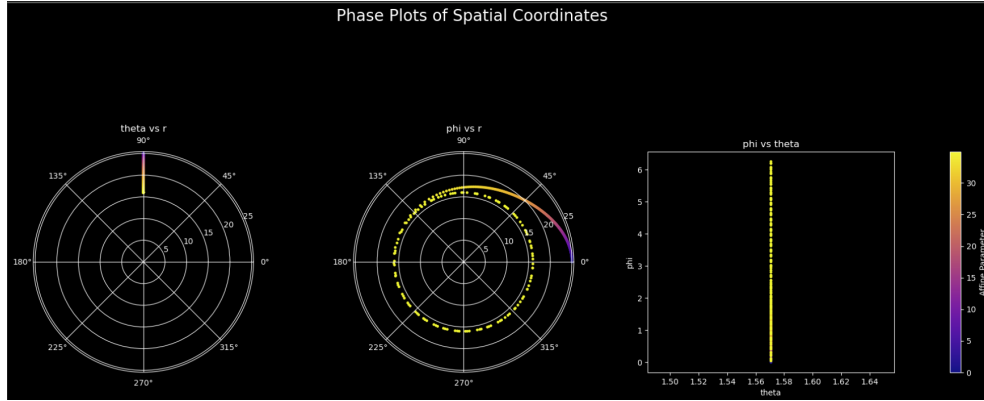
$$g_{\mu\nu} = \begin{bmatrix} -\left(1 - \frac{2Mx_2}{\Sigma}\right) & 0 & 0 & -\frac{2aMx_2 \sin^2(x_3)}{\Sigma} \\ 0 & \frac{\Sigma}{\Delta} & 0 & 0 \\ 0 & 0 & \Sigma & 0 \\ -\frac{2aMx_2 \sin^2(x_3)}{\Sigma} & 0 & 0 & \frac{(x_2^2 + a^2)^2 - a^2 \Delta \sin^2(x_3)}{\Sigma} \sin^2(x_3) \end{bmatrix}, \quad (10)$$

where

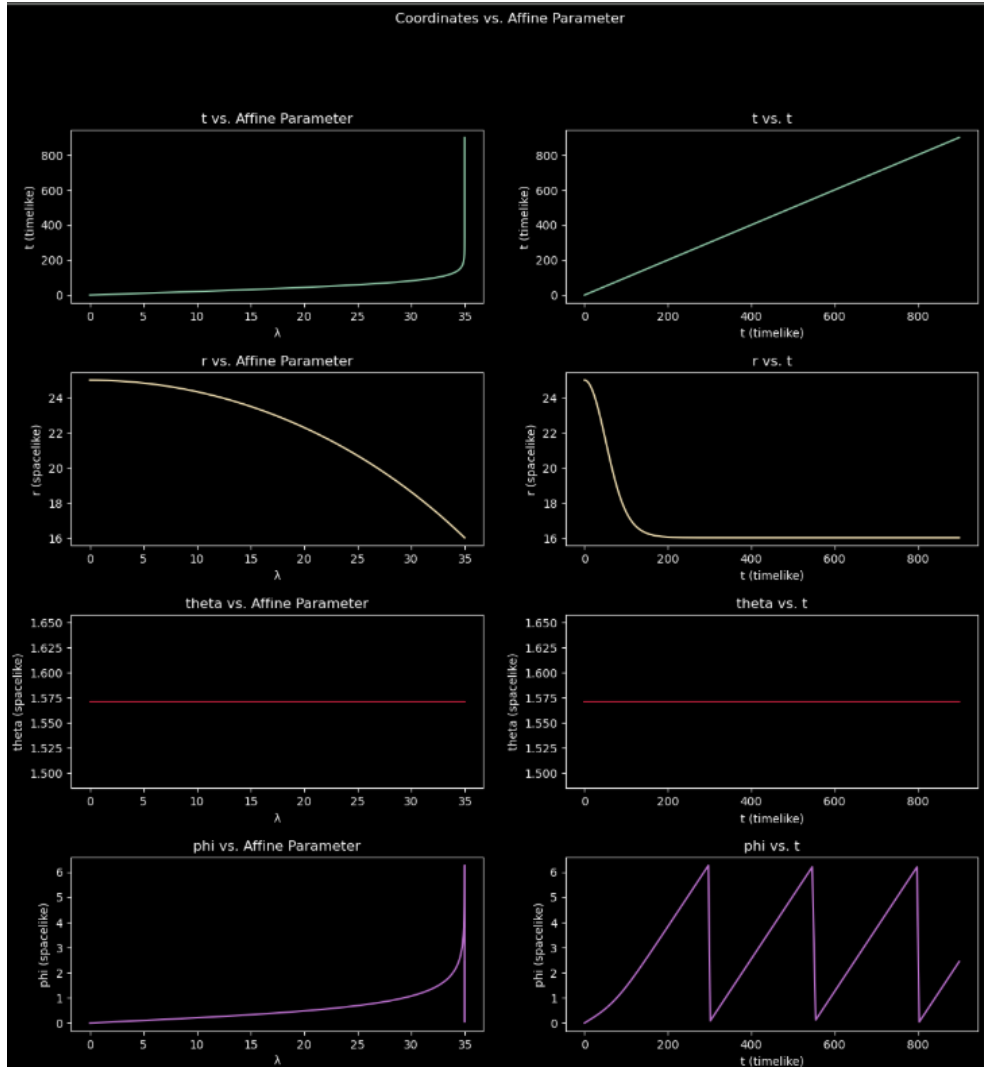
$$\Sigma = x_2^2 + a^2 \cos^2(x_3), \quad \Delta = x_2^2 - 2Mx_2 + a^2. \quad (11)$$

In this metric, M represents the mass of the black hole and a represents the spin parameter also known as the angular momentum per unit mass. The most important difference from the Schwarzschild metric is the off-diagonal term $g_{14} = g_{41}$. This term couples the timelike coordinate x_1 to the angular coordinate x_4 , which mathematically represents frame dragging.

Frame dragging means that the rotation of the black hole drags spacetime around with it, causing particles to gain angular motion even without an explicit angular momentum force applied. This effect is especially important in the ergosphere, a region outside the event horizon where spacetime is dragged so strongly by the rotating black hole that objects cannot remain stationary relative to a distant observer. Inside the ergosphere, particles are forced to co-rotate with the black hole, making it one of the more obvious physical signatures of Kerr spacetime.



(a) Phase plot of a massive particle moving in the Kerr metric.



(b) Coordinate evolution of the same massive particle in the Kerr metric.

Figure 11: Trajectory of a massive particle moving in the Kerr metric. The phase plot shows the spatial trajectory, while the coordinate plot shows how each coordinate evolves with affine parameter and coordinate time.

This plot shows a massive particle entering the ergosphere of the Kerr black hole. For this simulation, $M = 10$ and $a = 8$, so the outer event horizon is at

$$r_+ = M + \sqrt{M^2 - a^2} = 10 + \sqrt{100 - 64} = 16,$$

while the equatorial ergosphere boundary is at

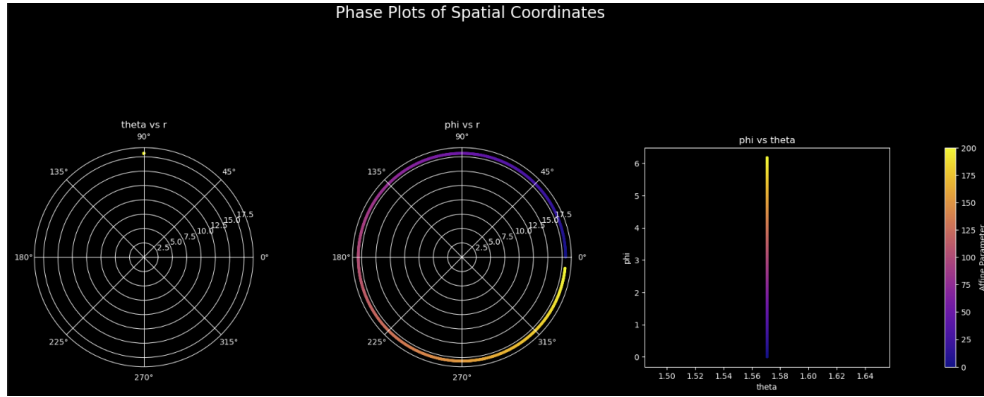
$$r_{\text{ergo}} = 2M = 20.$$

The particle begins outside the ergosphere at $r = 25$, but the trajectory moves inward and settles near $r \approx 16$. Since the ergosphere occupies the region

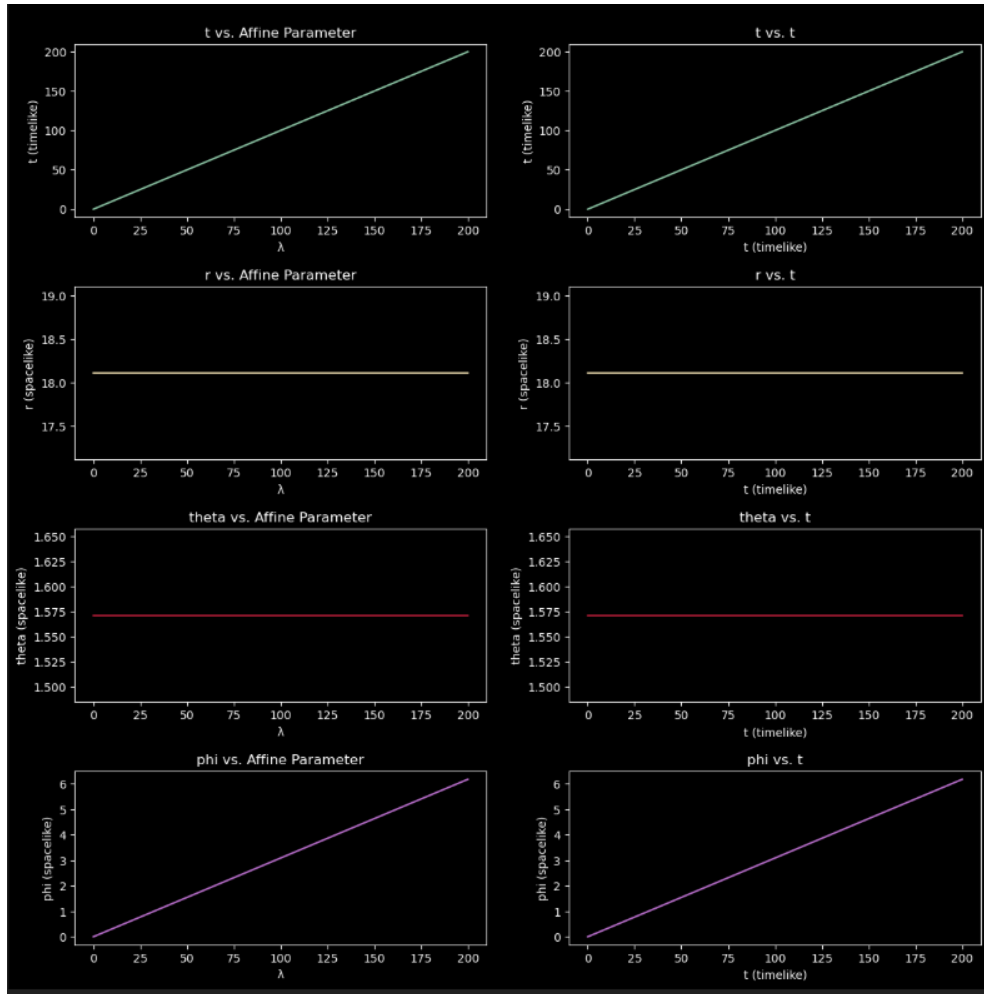
$$16 < r < 20$$

in the equatorial plane, the particle crosses into the ergosphere as its radial coordinate drops below $r = 20$. The circular motion in the ϕ direction shows the strong frame-dragging behavior of the Kerr metric, where the rotating black hole drags spacetime around with it and forces nearby particles to co-rotate.

The coordinate plots also show evidence of frame dragging. In the Kerr metric, the rotating black hole introduces an off-diagonal metric term $g_{x_1 x_4}$, which couples the timelike coordinate x_1 to the azimuthal coordinate x_4 . Because of this coupling, the particle's azimuthal coordinate changes rapidly even though no external angular force is applied. In the ϕ plots, this appears as an increasing angular coordinate, meaning the particle is being carried around the black hole by the rotating spacetime itself. This is the frame-dragging effect: the particle is not being pushed by a conventional force, but its geodesic bends in the ϕ -direction because the spacetime geometry is rotating.



(a) Phase plot of a massless particle moving in the Kerr metric.



(b) Coordinate evolution of the same massless particle in the Kerr metric.

Figure 12: Trajectory of a massless particle moving through the Kerr metric. The phase plot shows the spatial path, while the coordinate plot shows how the coordinates evolve along the geodesic.

For this Kerr photon simulation, the initial conditions were

$$x^\mu(0) = \left(0, 18.1, \frac{\pi}{2}, 0\right),$$

with initial spatial velocity

$$v^i = [0, 0, 0.0309].$$

The black hole parameters were

$$M = 10, \quad a = 8.$$

This trajectory represents a semi-stable photon orbit near a rotating black hole. In the Kerr metric, photon orbits depend on the spin of the black hole. Because the spacetime is rotating, frame dragging allows prograde photons to orbit closer to the event horizon than they could around a non-rotating black hole. The photon is not truly in a stable orbit, since photon orbits are generally unstable, but these initial conditions place it near a circular path where it can remain for a short time before small numerical or physical changes cause it to drift inward or outward.

3.4 FLRW Metric - Expanding Universe Spacetime

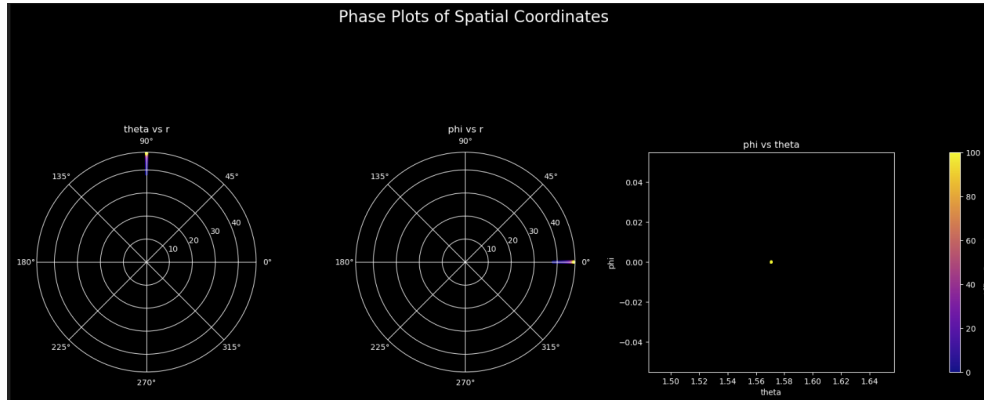
The FLRW metric describes a universe that is homogeneous and isotropic, meaning that on large scales the universe looks the same at every location and in every direction. Unlike the Schwarzschild or Kerr metrics, which describe spacetime around a black hole, the FLRW metric describes the geometry of the universe as a whole. In spherical-like comoving coordinates $\{t, r, \theta, \phi\}$, the spatially flat FLRW metric can be written as

$$g_{\mu\nu} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & a(t)^2 & 0 & 0 \\ 0 & 0 & a(t)^2 r^2 & 0 \\ 0 & 0 & 0 & a(t)^2 r^2 \sin^2(\theta) \end{bmatrix}. \quad (12)$$

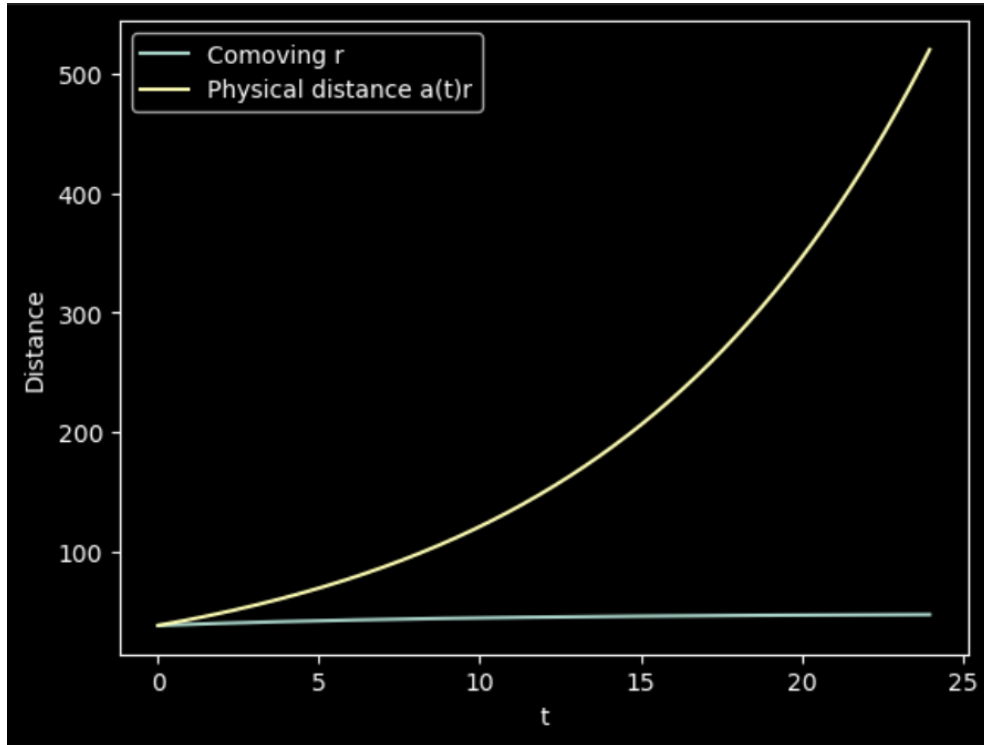
Here, $a(t)$ is the scale factor, which controls how distances expand with time. A particle can remain at a nearly fixed comoving coordinate r , while its physical distance still grows according to

$$d_{\text{phys}} = a(t)r. \quad (13)$$

Using our program, we tested particle motion in an expanding FLRW spacetime. The trajectory plot shows the particle remaining approximately stationary in comoving coordinates, while the expansion plot shows the physical distance increasing as the scale factor grows. This demonstrates the difference between motion through space and the expansion of space itself.



(a) Phase plot of a massive particle moving in the FLRW metric.



(b) Universe is expanding even though the coordinates remain unchanged..

Figure 13: A comparison of particle motion in an expanding FLRW space-time. The stationary trajectory shows that the particle remains at an approximately fixed comoving coordinate, while the expansion plot shows that its physical distance $a(t)r$ still grows as the scale factor increases. Together, these plots illustrate the difference between staying fixed in comoving coordinates and being carried farther away by the expansion of the universe.

3.5 Weird Metrics

Figures 14, 15, 16 showcase weird metric trajectories we tested using our code.

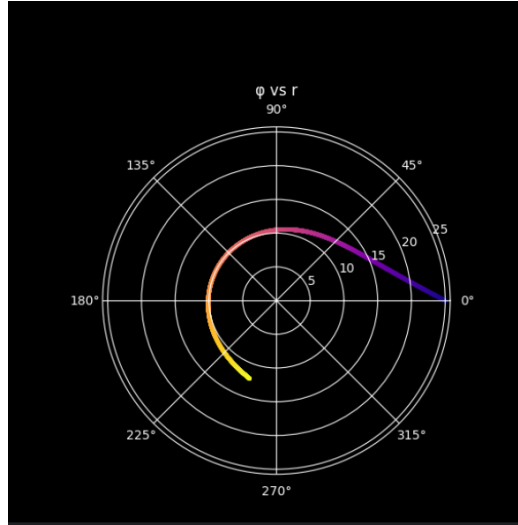


Figure 14: A massive particle scattered by a wormhole. This metric was essentially the Schwarzschild metric with $g_{tt} = -1$ so that the event horizon was traversable. This orbit is weird because the loop is nearly circular, and almost appears like a photon being lensed.

16.

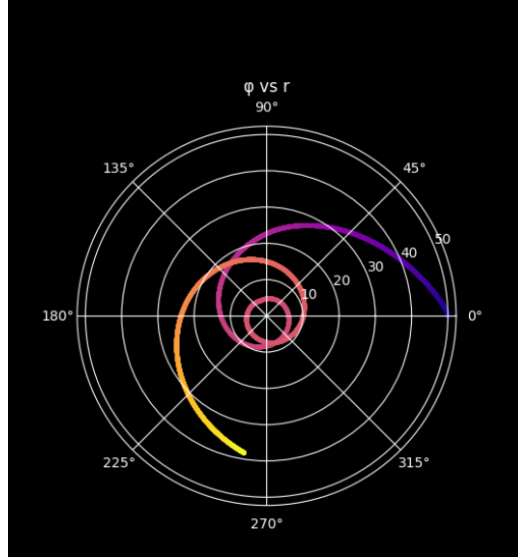


Figure 15: A massive particle in an esoteric alteration of the Schwarzschild metric I made which has off-diagonal terms that are sinusoidal. The resulting orbit is extremely strange yet pretty.

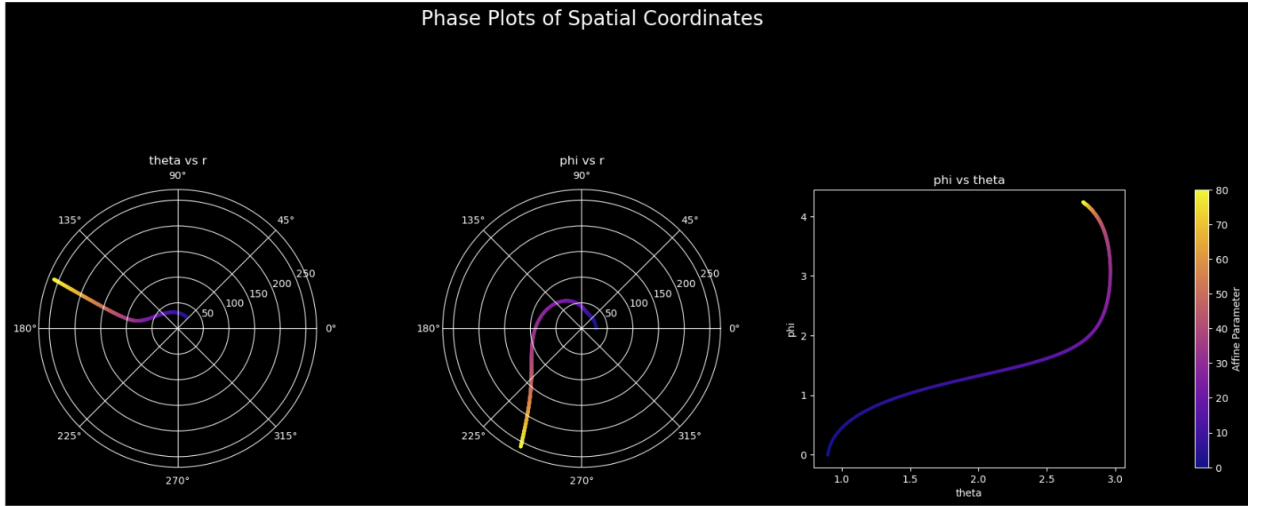


Figure 16: A particle moving through the Taub-NUT metric, an esoteric spacetime with a gravitomagnetic twisting term. The trajectory bends strangely through both polar and azimuthal directions, producing a short but unusual curved path before the particle falls inward.

4 Link to Simulation

Below is a link to the GitHub page of our simulation:

<https://github.com/04alexh/CompPhysicsProject>